

Real-Time Railway Track Monitoring System Using OpenCV

1. Railway Track Monitoring System – Write-Up

Railway track monitoring application involves monitoring the condition of the railway tracks for safety reasons, as it is important to ensure that any cracks or broken parts on the tracks are identified, and that there are no abnormal objects on the surface that could pose a risk to operations. This project aims to provide an OpenCV based prototype of railway track monitoring system for analysing the images of railway tracks to find possible defect regions in the track by using classical computer vision techniques. The system is capable of image resizing, grayscale conversion, contrast enhancement using CLAHE, Gaussian blurring, Canny edge detection, Probabilistic Hough Line Transform, binary thresholding and morphological opening and detection of contours. The candidate railway lines are outlined in green lines and contour areas that meet the chosen area limits are displayed in red bounding boxes (potential faults). It also records the processing time, latency and equivalent FPS and compares with the real-time requirements of 100ms equivalent latency and 15 FPS. We found 48 defects in the provided execution with 199.69 ms of processing time and 5.01 fps of equivalent speed, so the latency and processing-speed requirements did not pass for this execution. Edge Detection and Hough Transform are classical OpenCV methods that can be applied to railway line localization and pre-processing, but are sensitive to shadows, rust, illumination variations and complex backgrounds. Hence, the proposed future architecture is a combination of both, where the OpenCV will act as preprocessing and track/candidate localization, and the lightweight YOLO model will act as the final defect classification. In summary, the current implementation represents the OpenCV stage of a pipeline for railway monitoring, and can serve as a basis for the future integration of YOLO-based defect detection to implement more accurate, robust and timely railway defect monitoring.

1.1 Tools Used for Implementation

- Python
- OpenCV
- NumPy
- Time module for latency and FPS measurement
- Ultralytics YOLO (recommended for future defect classification)
- PyTorch for model training (future extension)
- ONNX Runtime or TensorRT for optimized edge inference (future extension)
- CUDA-enabled GPU or an edge device such as NVIDIA Jetson (future deployment)

2.Pseudocode

2.1. High-Level Railway Track Monitoring Pipeline Pseudocode

```
START

Start timer

Read railway track image

IF image cannot be opened
    Display error message
    STOP
END IF

Resize image
Copy image
Convert image to grayscale
Apply CLAHE
Apply Gaussian Blur
Apply Canny Edge Detection
Apply Hough Line Transform

IF lines are detected
    FOR each line
        Get line coordinates
        Calculate dx
        Calculate dy

        IF  $dy > dx \times 0.5$ 
            Draw line
        END IF
    END FOR
END IF

Apply binary threshold
Create morphological kernel
Apply morphological opening
Find contours

Set defect count to 0

FOR each contour
    Calculate contour area

    IF  $area > 50$  AND  $area < 5000$ 
        Find bounding rectangle
        Increase defect count
        Draw rectangle
        Display "Possible Defect"
    END IF
END FOR

Stop timer
Calculate elapsed time
Calculate latency

IF elapsed time  $> 0$ 
    Calculate equivalent FPS
ELSE
    Set FPS to 0
END IF

IF latency  $\leq 100$  ms
    Set latency result as PASS
ELSE
    Set latency result as FAIL
END IF

IF FPS  $\geq 15$ 
```

```

        Set speed result as PASS
ELSE
    Set speed result as FAIL
END IF

Display system title and results
Display input image
Display edge detection image
Display defect detection image
Save output image
Print defect count, processing time, and equivalent FPS
Wait for key press
Close all windows

STOP

```

3. Implementation and Output

3.1 Implementation Approach

The program reads the railway track image. Changes its size to 640 by 480 pixels. The image is turned to grayscale. Improved with CLAHE to make the local contrast better. Gaussian Blur is used to cut down noise before applying edge detection. The Probabilistic Hough Line Transform then finds line segments. The line orientation is checked with dx and dy. Then the program draws tracks that look like the real railway lines. To spot defects the blurred image is thresholded, cleaned with morphological opening and examined with contours. Contours that have an area more than 50 but less, than 5000 are marked as defects and a red bounding box is drawn around them. Finally the program measures latency and the equivalent FPS. Then shows if the required performance limits are met.

3.2 Code Listing

File: railway.py — OpenCV-Based Railway Track Monitoring and Possible Defect Detection

```

import cv2
import numpy as np
import time

start_time = time.time()

img = cv2.imread("railway_track.jpg")

if img is None:
    print("Error: Cannot open railway_track.jpg")
    print("Make sure the image is in the same folder as this program.")
    exit()

img = cv2.resize(img, (640, 480))
output = img.copy()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

enhanced = clahe.apply(gray)

blur = cv2.GaussianBlur(
    enhanced,
    (5, 5),
    0
)

```

```

edges = cv2.Canny(
    blur,
    50,
    150
)

lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi / 180,
    threshold=80,
    minLineLength=80,
    maxLineGap=20
)

if lines is not None:
    for line in lines:
        line = np.array(line).flatten()

        if len(line) >= 4:
            x1 = int(line[0])
            y1 = int(line[1])
            x2 = int(line[2])
            y2 = int(line[3])

            dx = abs(x2 - x1)
            dy = abs(y2 - y1)

            if dy > dx * 0.5:
                cv2.line(
                    output,
                    (x1, y1),
                    (x2, y2),
                    (0, 255, 0),
                    2
                )

_, binary = cv2.threshold(
    _,
    blur,
    80,
    255,
    cv2.THRESH_BINARY_INV
)

kernel = np.ones(
    (3, 3),
    np.uint8
)

binary = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel
)

contours, hierarchy = cv2.findContours(
    binary,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

defect_count = 0

for contour in contours:
    area = cv2.contourArea(contour)

    if area > 50 and area < 5000:
        x, y, w, h = cv2.boundingRect(contour)

        defect_count += 1

```

```

        cv2.rectangle(
            output,
            (x, y),
            (x + w, y + h),
            (0, 0, 255),
            2
        )

        cv2.putText(
            output,
            "Possible Defect",
            (x, max(y - 5, 15)),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5,
            (0, 0, 255),
            2
        )

    end_time = time.time()

    elapsed = end_time - start_time
    latency = elapsed * 1000

    if elapsed > 0:
        equivalent_fps = 1 / elapsed
    else:
        equivalent_fps = 0

    if latency <= 100:
        latency_result = "Latency: PASS"
    else:
        latency_result = "Latency: FAIL"

    if equivalent_fps >= 15:
        fps_result = "Speed: PASS"
    else:
        fps_result = "Speed: FAIL"

    cv2.putText(
        output,
        "Railway Track Monitoring System",
        (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )

    cv2.putText(
        output,
        "Possible Defects: " + str(defect_count),
        (10, 60),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 0, 255),
        2
    )

    cv2.putText(
        output,
        f"Processing Time: {latency:.2f} ms",
        (10, 90),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 255),
        2
    )

    cv2.putText(
        output,

```

```

        f"Equivalent FPS: {equivalent_fps:.2f}",
        (10, 120),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 255),
        2
    )

cv2.putText(
    output,
    latency_result,
    (10, 150),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (0, 255, 0),
    2
)

cv2.putText(
    output,
    fps_result,
    (10, 180),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (0, 255, 0),
    2
)

cv2.imshow("Input Railway Image", img)
cv2.imshow("Edge Detection", edges)
cv2.imshow("Possible Defects", output)

cv2.imwrite(
    "railway_monitoring_output.jpg",
    output
)

print("----- RAILWAY TRACK MONITORING RESULTS -----")
print("Possible defects detected:", defect_count)
print(f"Processing time: {latency:.2f} ms")
print(f"Equivalent FPS: {equivalent_fps:.2f}")

print("\n----- SYSTEM REQUIREMENTS -----")

if latency <= 100:
    print("Latency Requirement: PASS")
else:
    print("Latency Requirement: FAIL")

if equivalent_fps >= 15:
    print("Processing Speed Requirement: PASS")
else:
    print("Processing Speed Requirement: FAIL")

print("\nOutput saved as: railway_monitoring_output.jpg")

cv2.waitKey(0)
cv2.destroyAllWindows()

```

3.3 Input and Output Screenshots

Input test image and intermediate/final program outputs obtained from the provided implementation are shown below.

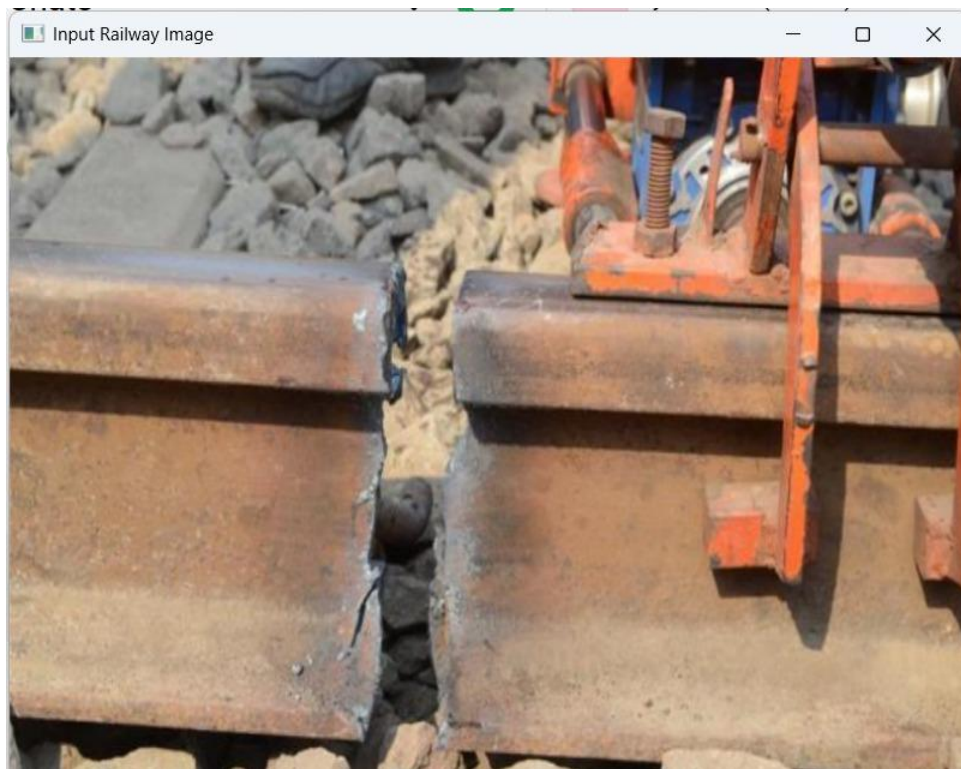


Figure 1: Input Railway Track Image

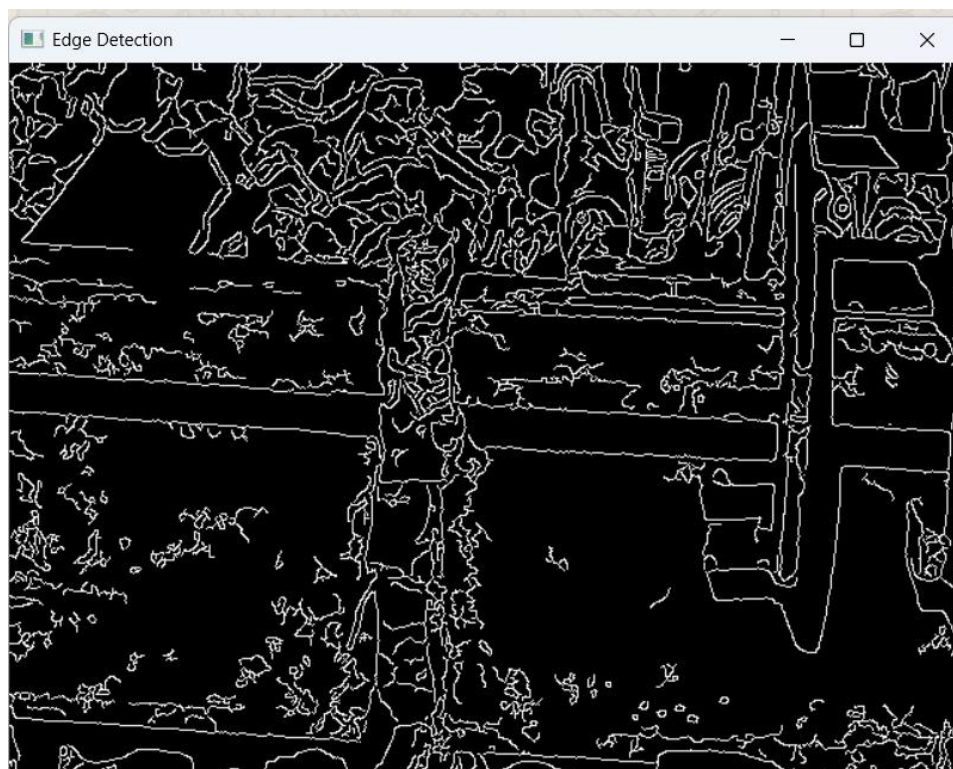


Figure 2: Canny Edge Detection Output

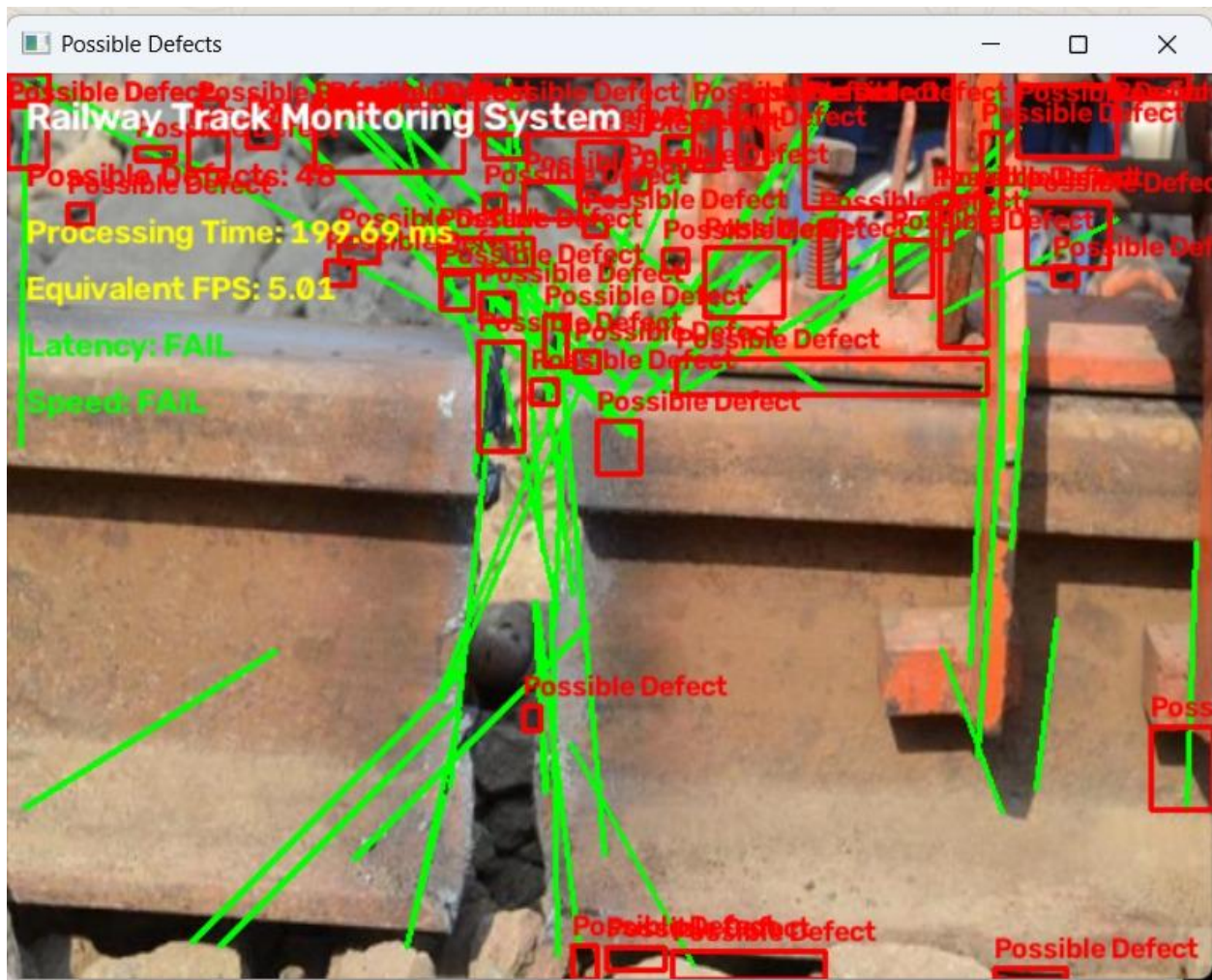


Figure 3: Possible Defect Detection Output

```
===== RESTART: C:/Users/pavan/OneDrive/Opencv/railway.py =====
----- RAILWAY TRACK MONITORING RESULTS -----
Possible defects detected: 48
Processing time: 199.69 ms
Equivalent FPS: 5.01

----- SYSTEM REQUIREMENTS -----
Latency Requirement: FAIL
Processing Speed Requirement: FAIL

Output saved as: railway_monitoring_output.jpg
```

Figure 4: Program Console Output and Performance Results

4. Results and Discussion

The provided execution demonstrates that the classical OpenCV pipeline was able to process the railway track image, display the Canny edge map, draw the candidate line segments and mark the regions based on the contours as possible defects. The 48 reported issues in the console, 199.69 ms of processing time and 5.01 fps were the results. The configured requirements are the following: Latency target should be $\leq 100\text{ms}$, Processing Speed target should be $\geq 15\text{FPS}$. But the measured run resulted in a latency FAIL and a speed FAIL. Also, the 48 detections are only candidate regions and should not be taken as actual defects because the implementation process classifies candidates that are on the contour as 'Possible Defect'. Other track structures, such as background texture, fasteners, shadows, etc. can lead to false detection.

5. Performance Summary

Metric	Requirement	Observed Result	Status
Possible defects detected	Not specified	48	Candidate detections
Latency	$\leq 100\text{ ms}$	199.69 ms	FAIL
Processing speed	$\geq 15\text{ FPS}$	5.01 FPS	FAIL

6. Conclusion

This project presents the Prototype of Railway track monitoring system based on OpenCV using some techniques such as CLAHE, Gaussian Blur, Canny Edge Detection, Probabilistic Hough Line Transform, Thresholding, Morphological Opening, and Contour Analysis. The system is able to track the railway edges and the candidate defect regions and generates a performance report of the system which is measurable. The execution provided was, however, not meeting the required real-time targets with 199.69 ms latency and 5.01 FPS. The contour based approach also yields a list of defect candidates, not defect classes. Improvements are required in the future to enhance the region-of-interest selection, the filtering, the optimization of the processing as well as the integration of a trained lightweight YOLO model for correct detection of cracks, rust and broken components and foreign objects.